Introduction
This is a technical interview for the position of a Gen AI engineer in Lovelytics.
We are more interested in your:

* Problem-solving methodology than perfect code - Show us how you think through challenges
* Architecture and design skills than implementation details - Demonstrate your ability to design scalable systems
* Practical judgment than theoretical knowledge - Make trade-offs that balance complexity, cost, and value

Guidelines:

Planning First
We strongly encourage you to:
* Think through your solution completely before coding
* Draft your architecture and document your approach
* Create diagrams to illustrate your system design
* Document your decisions and trade-offs

Technical Expectations

* Cloud-native solutions are highly valued
* Databricks experience is a strong plus
* Focus on scalable, maintainable systems

Practical Considerations
We understand you may have limitations:

* No financial investment required - use free tiers when possible
* No high-performance hardware needed - mock expensive compute operations
* Local development is acceptable - simulate cloud components locally
* Documentation over deployment - well-documented mocks are preferred over costly deployments

We kindly ask that the tech assessment be submitted within 3 business days.

Build a Financial AI Agent
The Scenario:
You're prototyping an AI assistant for financial fraud analysts. The system needs to understand natural language questions, query transaction databases, apply fraud detection models, and provide intelligent responses.

What We Want to See:

* How you architect an AI agent that combines data retrieval, ML predictions, and conversational responses
* Your approach to integrating structured transaction data with unstructured financial knowledge
* A working prototype that demonstrates real analytical workflows

Example Use Cases:

Your system should be able to handle diverse queries that combine data analysis, ML predictions, and knowledge retrieval:

Data Analysis Questions:
* "How many international transactions are in the dataset?"
* "What's the average transaction amount for fraudulent vs legitimate transactions?"
* "Which customers have the highest risk scores?"
* "How many transactions had more than 3 failed attempts in 24 hours?"
* "Compare purchase patterns between Gold and Silver membership tiers"

Predictive Questions:
* "Predict if this transaction is fraudulent: $1,250 purchase at electronics store, international, 3am, from a 2-month-old account"
* "What's the expected purchase amount for a 45-year-old customer with Platinum membership who made 20 transactions last month in the Home category?"
* "Which transactions from high-risk customers should we investigate first?"

Knowledge-Based Questions:
* "What are the common indicators of credit card fraud?"
* "Explain the KYC procedures for high-risk customers"
* "What are the key components of PCI DSS compliance?"
* "Summarize the machine learning approaches for fraud detection"

Complex Analytical Questions:
* "Which transactions look suspicious and why? Show me the top 5 with explanations"
* "What fraud patterns exist in international transactions vs domestic ones?"
* "Analyze customer CUST7823's transaction history and assess their fraud risk"

Your system should query data, apply fraud models, retrieve relevant financial knowledge, and provide intelligent, conversational responses. These questions are just an example and you shouldn't respond to all of them.

The Data

You are provided with three assets:

1. Transaction Fraud Dataset (`fraud_dataset.csv`)
- 100 transaction records with fraud labels
- **Target variable:** `fraud` (binary: 0 = legitimate, 1 = fraudulent)
- **Key features:** transaction amount, merchant category, device type, customer age, account age, transaction velocity, risk scores, IP reputation, failed transactions, international flag, address matching, CVV match, and more
- **Use case:** Build a binary classification model to predict fraudulent transactions

2. Customer Purchase Dataset (`product_purchase_dataset.csv`)
- 100 customer records with purchase behavior
- **Target variable:** `purchase_amount` (continuous value)
- **Key features:** customer demographics (age, gender, income), membership tier, transaction history, loyalty points, preferred categories, payment methods, satisfaction scores, and spending patterns
- **Use case:** Build a regression model to predict customer purchase amounts

3. Financial Knowledge Base (`financial_documents/` folder)
- 20 markdown documents covering:
  - Fraud detection patterns and indicators
  - Anti-money laundering (AML) procedures
  - KYC (Know Your Customer) requirements
  - Payment security and PCI DSS compliance
  - Transaction monitoring systems
  - Risk assessment methodologies
  - Regulatory compliance (GDPR, etc.)
  - Machine learning for fraud detection
  - Mobile banking security
  - And more financial services topics
- **Use case:** Provide your agent with domain knowledge to answer financial questions accurately

What You Should Deliver

Core Components
Your solution should include:

1. **Two ML Models:**
   - Fraud detection model (binary classification on `fraud_dataset.csv`)
   - Purchase prediction model (regression on `product_purchase_dataset.csv`)

2. **AI Agent/Conversational Interface:**
   - Understands natural language questions
   - Queries structured data (CSV files)
   - Applies ML models for predictions
   - Retrieves and cites information from financial documents

   - Provides intelligent, contextual responses

3. **Documentation:**
   - Architecture diagram or description
   - Model selection rationale and performance metrics
   - Key insights discovered from the data
   - Trade-offs and decisions made
   - What you'd improve with more time

Implementation Freedom
* Choose any tools, frameworks, or libraries
* Deliver as notebook, API, CLI, web app, or any format
* Use AI coding assistants if helpful (document your methodology)
* Focus on what showcases your strengths best

Reality Check
This is a **prototype**. We value:
- **Functional over perfect** - Show it works, don't over-engineer
- **Thinking over polish** - Demonstrate your problem-solving approach
- **Substance over features** - Depth in key areas beats breadth

Live Demo
Be prepared to demonstrate your solution during the technical interview. You'll walk through your system, explain your decisions, and answer questions about your approach.